

Архітектура шарів, бізнес-логіка та життєвий цикл компонентів (DI + IoC)

Лекція 3

Зміст лекції

- Архітектура шарів та зв'язування компонентів
- Моделювання бізнес-логіки та бізнес-правила
- Обробка доменних винятків у REST API
- Патерни проєктування за допомогою Spring DI
- Послаблення зв'язності через доменні події
- Життєвий цикл бінів та Scopes
- Ізольоване модульне тестування бізнес-логіки

Наслідки концентрації логіки у веб-контролері

Що відбувається із кодовою базою, якщо валідацію IP-адрес, розрахунок бітрейту та збереження камер розмістити безпосередньо в методах контролера?

Архітектура шарів та зв'язування компонентів

Розподіл відповідальності у класичній тришаровій архітектурі:



- **Спрямованість залежностей:** Кожен шар взаємодіє лише із сусіднім рівнем через інтерфейси.

Зона відповідальності сервісного шару (@Service)

Анотація `@Service` позначає клас як компонент бізнес-логіки застосунку:

- Сервіс координує взаємодію між доменними об'єктами, репозиторіями та зовнішніми шлюзами.
- Сервіси реєструються як біни з областю видимості Singleton. Вони не повинні містити змінюваного стану користувача, щоб уникати конфліктів у багатопотоковому середовищі.
- Саме на рівні сервісного шару визначаються межі бізнес-операцій та координація супутніх дій.

Контрактне проєктування через інтерфейси

Контролер взаємодіє виключно з інтерфейсом сервісу, що забезпечує послаблення зв'язності:

```
public interface CameraService {  
    CameraResponse registerCamera(RegisterCameraCommand command);  
    CameraResponse updateStatus(String id, UpdateCameraStatusCommand command);  
    CameraResponse getCameraById(String id);  
    List<CameraResponse> getAllCameras();  
}
```

Репозиторій як інтерфейсний контракт

До підключення реляційної бази даних збереження даних організовується через інтерфейс сховища:

```
public interface CameraRepository {  
    Camera save(Camera camera);  
    Optional<Camera> findById(String id);  
    List<Camera> findAll();  
    boolean existsById(String id);  
    boolean existsByIpAddress(String ipAddress);  
    void deleteById(String id);  
}
```

Сервіс залежить лише від контракту `CameraRepository`. Реалізація на цій лекції використовує потокобезпечні колекції (`ConcurrentHashMap`), що дозволяє без змін бізнес-коду замінити її на БД в наступній лекції.

Зв'язування шарів: Spring IoC та DI

Spring IoC-контейнер автоматично зв'язує компоненти через конструктор:

```
@RestController
@RequestMapping("/api/cameras")
public class CameraController {
    private final CameraService cameraService;

    public CameraController(CameraService cameraService) {
        this.cameraService = cameraService;
    }
}
```

- Constructor Injection забезпечує незмінність (`final`) полів та повну готовність об'єкта до роботи.
- Створенням і зв'язуванням екземплярів керує Spring IoC.

Автоматичне сканування: @Component та стереотипи

Основний механізм реєстрації компонентів для власного коду застосунку:

- **Автоматичне виявлення:** Під час запуску Spring сканує пакети, починаючи з кореневого пакета `@SpringBootApplication`.
- **Спеціалізовані анотації:**
 - `@Component` — базовий загальний стереотипний компонент.
 - `@Service` — компонент бізнес-логіки та сценаріїв використання.
 - `@Repository` — компонент доступу до даних (трансляція винятків сховища).
 - `@RestController` — обробник веб-запитів та серіалізація відповідей у JSON.

Декларативна реєстрація: @Configuration та @Bean

- Використовується, коли клас неможливо анотувати `@Component` (наприклад, класи з JDK або зовнішніх бібліотек).
- Метод з анотацією `@Bean` повертає екземпляр, який Spring реєструє в контексті.
- Параметр `proxyBeanMethods = false` вимикає CGLIB-проксі над класом конфігурації для прискорення запуску.

```
@Configuration(proxyBeanMethods = false)
public class AppConfig {
    @Bean
    public Clock appClock() {
        return Clock.systemUTC();
    }
}
```

Пошук бінів за типом та проблема неоднозначності

Як Spring визначає, який саме клас впровадити для контракту інтерфейсу:

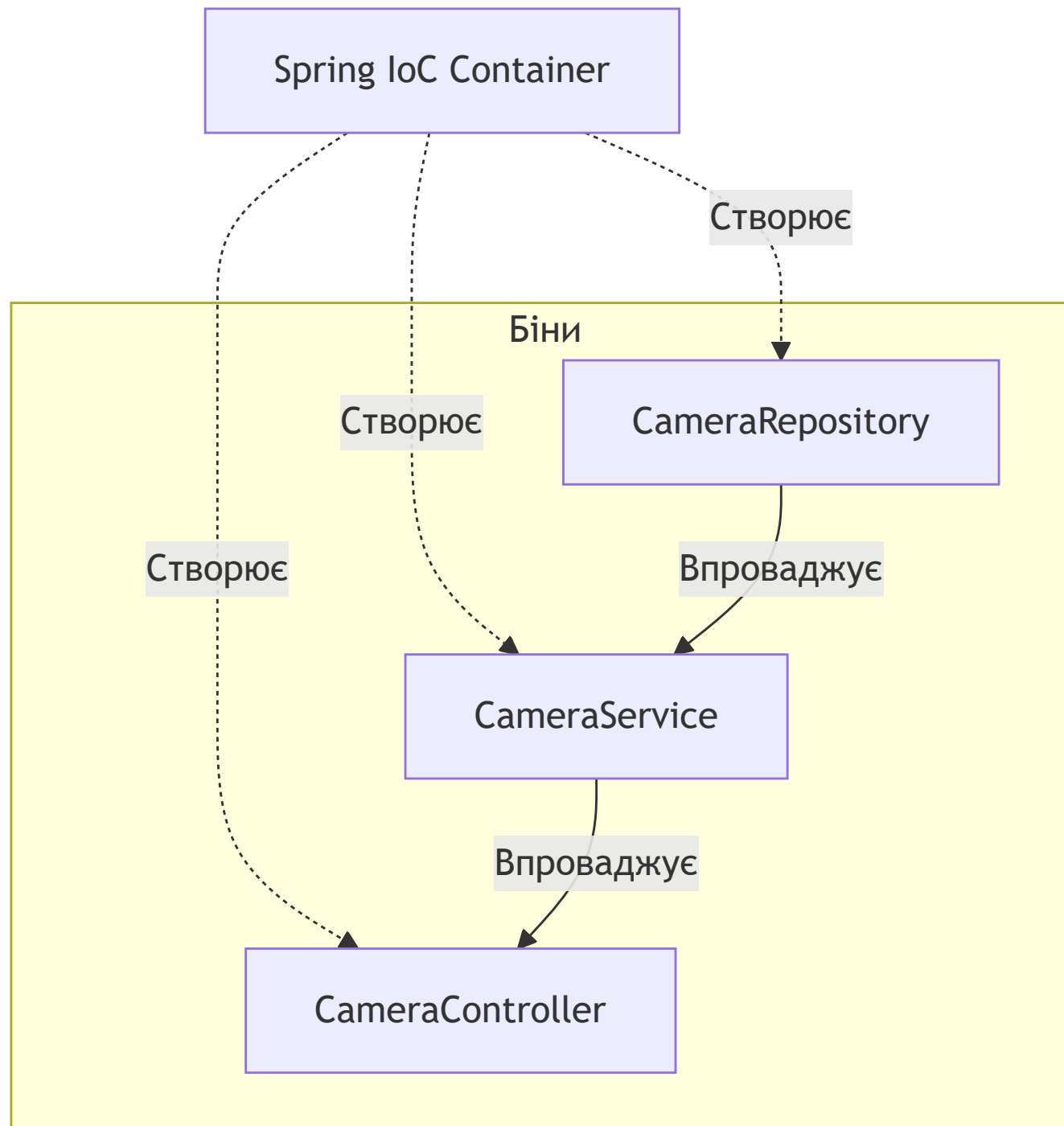
- **Пошук за типом (Autowiring by Type):**
 - За замовчуванням Spring перевіряє інтерфейс залежності (наприклад, `CameraService`).
 - Якщо в контексті зареєстровано рівно один бін відповідного типу (`CameraServiceImpl`), контейнер успішно зв'язує його.
- **Конфлікт множинних реалізацій:**
 - Якщо в контексті одночасно зареєстровано два або більше бінів одного типу (наприклад, `CameraServiceImpl` та `MockCameraService`).
 - Spring зупиняє запуск застосунку з винятком `NoUniqueBeanDefinitionException` .

Розв'язання неоднозначності: @Primary та @Qualifier

- **@Primary** : Визначає пріоритетний бін за замовчуванням при наявності кількох реалізацій інтерфейсу.
- **@Qualifier** : Вказує точне ім'я необхідного біна в точці впровадження залежності.

```
@RestController
public class CameraController {
    private final CameraService cameraService;

    public CameraController(@Qualifier("cameraServiceImpl") CameraService cameraService) {
        this.cameraService = cameraService;
    }
}
```



Моделювання бізнес-логіки та бізнес-правила

Валідація даних у корпоративних системах розділяється на два принципово різні рівні:

1. Валідація формату вхідних даних:

- Перевіряється на рівні контролера анотаціями Jakarta Validation.
- Питання: *Чи є рядок непорожнім? Чи коректний синтаксис IP-адреси?*

2. Бізнес-валідація та дотримання правил:

- Перевіряється виключно в сервісному або доменному шарі.
- Питання: *Чи вільна IP-адреса в мережі? Чи дозволено перехід камери в режим обслуговування?*

Бізнес-правила у сервісному шарі

Бізнес-перевірки потребують доступу до поточного стану системи, історії дій та зв'язаних даних у сховищі:

- Умова або обмеження, яке має беззастережно виконуватися для сутності протягом усього її життєвого циклу.
- Перевірка вимагає запиту до сховища даних (наприклад, перевірка зайнятості мережевої адреси).
- Виконується виключно в сервісному або доменному шарі перед зміною стану.

Перевірка бізнес-правила у сервісі

Перевірка унікальності ідентифікатора та IP-адреси перед реєстрацією камери:

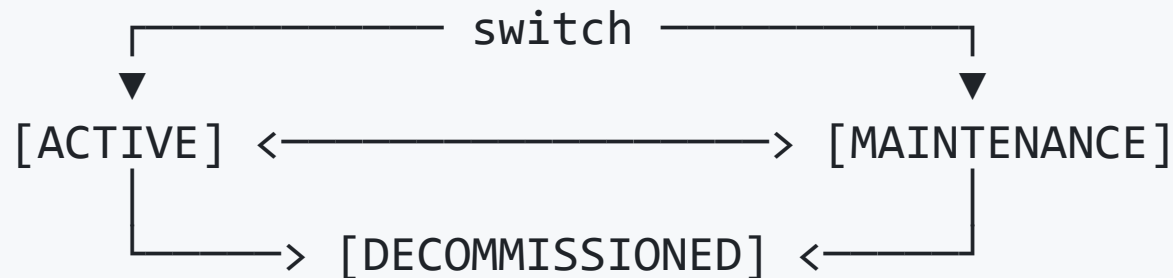
```
@Service
public class CameraServiceImpl implements CameraService {
    private final CameraRepository cameraRepository;
    // ...

    @Override
    public CameraResponse registerCamera(RegisterCameraCommand command) {
        if (cameraRepository.existsById(command.id())) {
            throw new DuplicateCameraException("Camera with ID '" + command.id() + "' already exists");
        }
        if (cameraRepository.existsByIpAddress(command.ipAddress())) {
            throw new DuplicateCameraException("Camera with IP address '" + command.ipAddress() + "' is already registered");
        }
        // подальша обробка та збереження
    }
}
```


Управління станами: концепція скінченного автомата

Життєвий цикл пристрою моделюється як **скінченний автомат**:

- **Стан**: Камера перебуває в одному фіксованому стані зі скінченної множини.
- **Переходи**: Зміна стану дозволена лише за строго визначеними правилами.



Правило проєктування: Спроба здійснити непередбачений перехід порушує бізнес-правило і має негайно блокуватися винятком.

Моделювання переходів між станами

Правила переходів скінченного автомата визначають допустимий життєвий цикл камери:

- **Дозволені переходи:**
 - `ACTIVE` переходить у `MAINTENANCE` або `DECOMMISSIONED`.
 - `MAINTENANCE` переходить у `ACTIVE` або `DECOMMISSIONED`.
- **Термінальний стан:**
 - `DECOMMISSIONED` є кінцевим станом; виведену з експлуатації камеру не можна активувати повторно.
- **Гарантія цілісності:** Унеможлиблюється поява суперечливих станів обладнання.

Реалізація FSM: метод canTransitionTo

```
public enum CameraStatus {  
    ACTIVE,  
    MAINTENANCE,  
    DECOMMISSIONED;  
  
    public boolean canTransitionTo(CameraStatus next) {  
        return switch (this) {  
            case ACTIVE -> next == MAINTENANCE || next == DECOMMISSIONED;  
            case MAINTENANCE -> next == ACTIVE || next == DECOMMISSIONED;  
            case DECOMMISSIONED -> false;  
        };  
    }  
}
```

- Метод `canTransitionTo` інкапсулює граф переходів безпосередньо в домені.
- Якщо перехід неприпустимий, сервіс генерує `InvalidCameraStateException`.

Захист переходів FSM у сервісному шарі

Сервіс викликає `canTransitionTo` перед збереженням зміненого стану в репозиторії:

```
@Override
public CameraResponse updateStatus(String id, UpdateCameraStatusCommand command) {
    Camera camera = cameraRepository.findById(id)
        .orElseThrow(() -> new CameraNotFoundException("Camera with ID '" + id + "' not found"));

    CameraStatus currentStatus = camera.getStatus();
    CameraStatus targetStatus = command.targetStatus();

    if (!currentStatus.canTransitionTo(targetStatus)) {
        throw new InvalidCameraStateException(
            "Illegal state transition for camera '" + id + "' from '" + currentStatus + "' to '" + targetStatus);
    }

    camera.setStatus(targetStatus);
    Camera updated = cameraRepository.save(camera);
    return toResponse(updated);
}
```

Обробка доменних винятків у REST API

Порушення бізнес-правил не повинні призводити до необроблених системних помилок зі статусом 500:

- **Доменні винятки (Domain Exceptions):** Спеціалізовані класи-спадкоємці `RuntimeException`, що відображають конкретну проблему предметної області.
- **Незмінність контракту:** Сервіс генерує бізнес-виняток, а веб-шар через `@RestControllerAdvice` транслює його у стандартизований HTTP-статус та тіло `ProblemDetail`.

```
[Сервіс: CameraService] — викидає —> InvalidCameraStateException
                                   |
[ControllerAdvice] — перехоплює —> HTTP 422 Unprocessable Entity
```

Створення ієрархії доменних винятків

Створення чіткої ієрархії винятків спрощує їх подальшу маршрутизацію у глобальному обробнику:

```
// Базовий абстрактний виняток домену
public abstract class DomainException extends RuntimeException {
    protected DomainException(String message) {
        super(message);
    }
}

// Порушення моделі переходів станів (FSM)
public class InvalidCameraStateException extends DomainException {
    public InvalidCameraStateException(String message) {
        super(message);
    }
}
```

Інтеграція доменних винятків з ProblemDetail

Глобальний обробник транслює доменні винятки у стандартизований формат RFC 9457:

- Анотація `@RestControllerAdvice` перехоплює помилки з усіх контролерів застосунку.
- Семантичні HTTP-статуси:
 - `422 Unprocessable Entity` — порушення бізнес-правила або некоректний перехід стану (`InvalidCameraStateException`).
 - `409 Conflict` — дублювання ресурсу (`DuplicateCameraException`).
 - `404 Not Found` — відсутність запитуваної камери (`CameraNotFoundException`).

Патерни проєктування за допомогою Spring DI

У реальних проєктах часто виникає потреба застосовувати різні алгоритми обробки (наприклад, розрахунок бітрейту для різних типів роздільної здатності камери: HD, SD, 4K).

Традиційне процедурне рішення:

```
// Порушення принципу Open/Closed (OCP)
if ("HD".equalsIgnoreCase(resolutionType)) {
    return 4096;
} else if ("SD".equalsIgnoreCase(resolutionType)) {
    return 1024;
} else if ("4K".equalsIgnoreCase(resolutionType)) {
    return 16384;
}
```

Кожне нове правило змушує змінювати вже перевірений та протестований сервіс.

Патерн «Стратегія» через механізми Spring

Як додати в систему підтримку нових форматів трансляції без редагування жодного рядка коду в основному сервісі?

Як контейнер Spring дозволяє реалізувати патерн «Стратегія» автоматично через впровадження колекцій бінів?

Впровадження колекцій бінів у Spring

- **Автоматичний збір:** Фреймворк знаходить усі зареєстровані класи з `@Component`, які реалізують інтерфейс, і збирає їх у колекцію.

```
public interface ResolutionStrategy {  
    String getResolutionType();  
    int calculateTargetBitrateKbps();  
}  
  
@Component  
public class HighResolutionStrategy implements ResolutionStrategy {  
    @Override  
    public String getResolutionType() { return "HD"; }  
    @Override  
    public int calculateTargetBitrateKbps() { return 4096; }  
}
```

Архітектура впровадження колекції стратегій

- Параметр конструктора `List<ResolutionStrategy>` автоматично отримує всі зареєстровані реалізації без `@Qualifier`.
- У тілі конструктора отриманий список перетворюється на `Map<String, ResolutionStrategy>` для швидкого доступу за типом.

```
public CameraServiceImpl(  
    CameraRepository cameraRepository,  
    ApplicationEventPublisher eventPublisher,  
    List<ResolutionStrategy> strategyList) {  
    this.cameraRepository = cameraRepository;  
    this.eventPublisher = eventPublisher;  
    this.strategies = strategyList.stream()  
        .collect(Collectors.toMap(  
            s -> s.getResolutionType().toUpperCase(),  
            Function.identity()));  
}
```

Виконання обраної стратегії

Метод сервісу знаходить відповідну стратегію та делегує їй виконання розрахунку:

```
String resolutionType = command.resolutionType().toUpperCase();
ResolutionStrategy strategy = strategies.get(resolutionType);

if (strategy == null) {
    throw new InvalidCameraStateException("Unsupported resolution type: " + command.resolutionType());
}

int targetBitrate = strategy.calculateTargetBitrateKbps();
Camera camera = new Camera(
    command.id(), command.location(), command.ipAddress(),
    resolutionType, targetBitrate, CameraStatus.ACTIVE);
```

Принцип Open/Closed

Архітектура на базі списку стратегій реалізує принцип відкритості/закритості (OCP):

- **Відкритість для розширення:** Для додавання формату UltraHD достатньо створити один новий клас з `@Component`.
- **Закритість для змін:** Клас `CameraServiceImpl` не зазнає жодних змін.
- **Чистота коду:** Повністю усувається розростання умовних операторів та мінімізується ризик регресії.

Послаблення зв'язності через доменні події

У бізнес-сценаріях після реєстрації камери часто потрібно виконати супутні дії (побічні ефекти):

- Записати подію у журнал безпеки та аудиту.
- Оновити статистику вузлів системи.
- Ініціалізувати канал відеопотоку.

Якщо викликати всі ці дії послідовно всередині `CameraService`, він стане надмірно перевантаженим залежностями на супутні сервіси.

Зв'язування компонентів без прямої залежності

Як сповістити підсистему аудиту про реєстрацію камери, щоб сервіс камер не мав прямого зв'язку з журналом аудиту?

Який механізм Spring Boot та Spring Modulith дозволяє реалізувати модель Publisher-Subscriber усередині застосунку?

Компоненти шини подій Spring

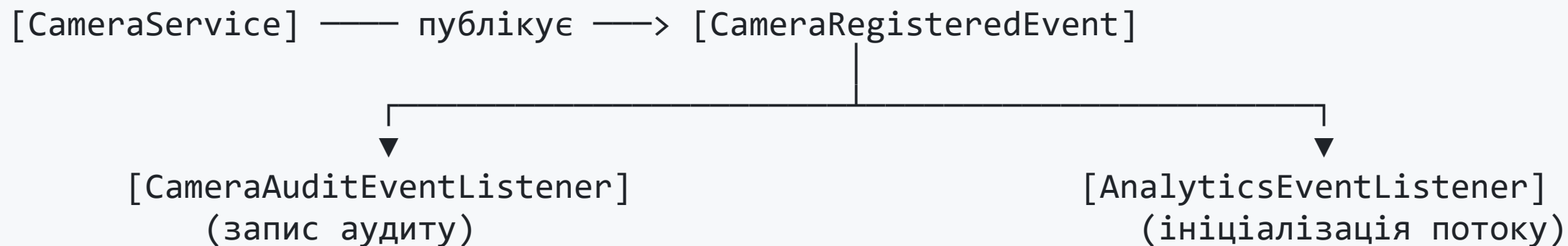
Spring надає вбудовану модель подій за допомогою компонента

`ApplicationEventPublisher` :

1. **Доменна подія (Event):** Незмінний об'єкт (Java Record), що фіксує факт події, яка вже відбулася у системі.
2. **Видавець (Publisher):** Бізнес-сервіс впроваджує `ApplicationEventPublisher` та публікує подію після фіксації сутності.
3. **Слухач (Listener):** Бін підписується на отримання події за допомогою анотацій обробки подій.

Топологія взаємодії через події

Слухачі отримують подію незалежно один від одного, що забезпечує послаблення зв'язності:



- Видавець події не має прямих залежностей на підсистеми аудиту чи аналітики.
- Нові слухачі додаються без модифікації коду сервісу камери.

Публікація доменної події

Оголошення події у вигляді Java Record:

```
public record CameraRegisteredEvent(  
    String cameraId,  
    String location,  
    String ipAddress  
) {}
```

Публікація події з бізнес-сервісу після збереження:

```
Camera saved = cameraRepository.save(camera);  
  
eventPublisher.publishEvent(new CameraRegisteredEvent(  
    saved.getId(),  
    saved.getLocation(),  
    saved.getIpAddress()  
));
```

Обробка подій у Spring Modulith

Spring Modulith надає спеціалізовану анотацію `@ApplicationModuleListener` :

- Поєднує транзакційність, асинхронне виконання та гарантію доставки між модулями моноліту.
- Дозволяє зберігати чистоту кордонів між доменними підсистемами.

```
@Component
public class CameraAuditEventListener {
    private static final Logger log = LoggerFactory.getLogger(CameraAuditEventListener.class);

    @ApplicationModuleListener
    public void onCameraRegistered(CameraRegisteredEvent event) {
        log.info("Audit log: New camera registered with id={}, location={}, ip={}",
            event.cameraId(), event.location(), event.ipAddress());
    }
}
```

Механіка роботи @ApplicationModuleListener

Анотація оптимізована для міжмодульної надійності:

- **Транзакційна фаза (AFTER_COMMIT):**
 - Слухач спрацьовує лише після успішної фіксації транзакції. Якщо збереження камери викликало помилку, аудит не запишеться помилково.
- **Асинхронний режим:**
 - Обробник не сповільнює HTTP-відповідь клієнту, оскільки запускається в окремому пулі потоків.
- **Архітектурна верифікація:**
 - `ApplicationModules.of(CctvApplication.class).verify()` гарантує, що слухач не звертається до внутрішніх пакетів інших модулів.

Життєвий цикл бінів та Scopes

За замовчуванням усі біни у Spring мають область видимості **Singleton**:

- **Один екземпляр на весь застосунок:** Створюється під час запуску контексту і використовується повторно всіма потоками.
- **Вимога до стану:** Singleton-біни повинні бути строго **Stateless** (не містити змінюваних полів сесії користувача).
- **Небезпека змінюваного стану (Mutable State):** Якщо додати в `@Service` звичайне поле `private String lastAccessedCameraId`, запити різних клієнтів перезаписуватимуть це поле одночасно, спричиняючи стан перегонів (Race Condition).

Призначення анотації @PostConstruct

Анотація `@PostConstruct` позначає метод, який виконується один раз після завершення впровадження всіх залежностей:

- **Чому не конструктор:** У момент роботи конструктора залежності або проксі ще можуть бути не до кінця ініціалізовані контейнером.
- **Типові сценарії використання:**
 - Завантаження первинних конфігурацій або довідників при старті.
 - Перевірка коректності зареєстрованих стратегій чи параметрів.
 - Ініціалізація внутрішніх з'єднань.

Ініціалізація через @PostConstruct

Логування та перевірка кількості доступних стратегій після створення сервісу:

Призначення анотації @PreDestroy

Анотація `@PreDestroy` вказує метод, що викликається безпосередньо перед знищенням біна при зупинці контексту:

- **Концепція Graceful Shutdown:** Застосунок не повинен переривати обробку даних раптово при отриманні сигналу завершення роботи (SIGTERM).
- **Типові сценарії використання:**
 - Гарантоване закриття відкритих каналів відеопотоків.
 - Скидання буферизованих у пам'яті журналів аудиту на диск.
 - Звільнення системних ресурсів.

Звільнення ресурсів через @PreDestroy

Коректне закриття з'єднань перед вимиканням JVM:

```
@Service
public class CameraServiceImpl implements CameraService {
    private static final Logger log = LoggerFactory.getLogger(CameraServiceImpl.class);
    // ...

    @PreDestroy
    public void cleanup() {
        log.info("CameraService shutting down, releasing resources");
    }
}
```


Пастка невідповідності областей видимості (Scope Mismatch)

Якщо бін повинен створюватися заново при кожному зверненні, його позначають `@Scope("prototype")` :

- **Суть проблеми:** Singleton створюється лише один раз під час запуску контексту. Якщо впровадити Prototype у Singleton через конструктор, він зафіксується назавжди і не буде створюватися заново.
- **Небезпека:** Помилкове очікування нового стану призводить до прихованих помилок спільного використання екземпляра різними потоками.

Оголошення Prototype-компонента

Сам компонент, призначений для багаторазового створення, обов'язково позначається prototype:

```
@Component
@Scope("prototype")
public class CameraStreamWorker {
    private static final Logger log = LoggerFactory.getLogger(CameraStreamWorker.class);

    public void initializeStream(String cameraId) {
        log.info("Initialized dedicated camera stream worker for camera {}", cameraId);
    }
}
```

- **Застереження:** Без анотації `@Scope("prototype")` клас залишиться Singleton, і будь-яка фабрика повертатиме один спільний екземпляр.

Рішення: Використання ObjectProvider

Для отримання нового екземпляра Prototype на кожну вимогу застосовують

ObjectProvider<T> :

```
@Service
public class CameraStreamService {
    private final ObjectProvider<CameraStreamWorker> workerProvider;

    public CameraStreamService(ObjectProvider<CameraStreamWorker> workerProvider) {
        this.workerProvider = workerProvider;
    }

    public void startStreaming(String cameraId) {
        // Отримуємо новий окремий екземпляр на кожний виклик
        CameraStreamWorker worker = workerProvider.getObject();
        worker.initializeStream(cameraId);
    }
}
```

Ізольоване модульне тестування бізнес-логіки

Правильно спроектований сервіс бізнес-логіки:

1. Отримує всі залежності виключно через конструктор.
2. Не залежить від класів веб-рівня (`HttpServletRequest` , `ResponseEntity`).
3. Працює з інтерфейсами репозиторіїв та бізнес-стратегій.

Анатомія тесту з Mockito

Для ізоляції залежностей використовується розширення `MockitoExtension` :

```
@ExtendWith(MockitoExtension.class)
class CameraServiceImplTest {

    @Mock
    private CameraRepository cameraRepository;

    @Mock
    private ApplicationEventPublisher eventPublisher;

    private CameraServiceImpl cameraService;

    @BeforeEach
    void setUp() {
        List<ResolutionStrategy> strategies = List.of(
            new HighResolutionStrategy(),
            new StandardResolutionStrategy());
        cameraService = new CameraServiceImpl(cameraRepository, eventPublisher, strategies);
    }
}
```

Тестування успішного бізнес-сценарію

Успішний тест перевіряє виклик методів залежностей та повернений результат:

```
@Test
void shouldRegisterCameraSuccessfully() {
    RegisterCameraCommand cmd = new RegisterCameraCommand("CAM-1", "Main Entrance", "192.168.1.10", "HD");
    when(cameraRepository.existsById("CAM-1")).thenReturn(false);
    when(cameraRepository.existsByIpAddress("192.168.1.10")).thenReturn(false);
    when(cameraRepository.save(any(Camera.class))).thenAnswer(inv -> inv.getArgument(0));

    CameraResponse response = cameraService.registerCamera(cmd);

    assertNotNull(response);
    assertEquals("CAM-1", response.id());
    assertEquals(4096, response.targetBitrateKbps());
    verify(cameraRepository).save(any(Camera.class));
    verify(eventPublisher).publishEvent(any(CameraRegisteredEvent.class));
}
```

Тестування викидання бізнес-винятків

Перевірка дотримання бізнес-правил тестується через `assertThrows` :

```
@Test
void shouldThrowDuplicateCameraExceptionWhenIdAlreadyExists() {
    RegisterCameraCommand cmd = new RegisterCameraCommand("CAM-1", "Main Entrance", "192.168.1.10", "HD");
    when(cameraRepository.existsById("CAM-1")).thenReturn(true);

    assertThrows(DuplicateCameraException.class, () -> {
        cameraService.registerCamera(cmd);
    });

    verify(cameraRepository, never()).save(any());
    verify(eventPublisher, never()).publishEvent(any());
}
```

Тестування правил FSM: невалідний перехід

Перевірка відхилення некоректної зміни стану сутності через `assertThrows` :

```
@Test
void shouldThrowInvalidCameraStateExceptionOnIllegalTransition() {
    Camera decommissioned = new Camera("CAM-3", "Warehouse", "192.168.1.30", "SD", 1024, CameraStatus.DECOMMISSIONED);
    when(cameraRepository.findById("CAM-3")).thenReturn(Optional.of(decommissioned));

    UpdateCameraStatusCommand cmd = new UpdateCameraStatusCommand(CameraStatus.ACTIVE);

    assertThrows(InvalidCameraStateException.class, () -> {
        cameraService.updateStatus("CAM-3", cmd);
    });

    verify(cameraRepository, never()).save(any());
}
```


Висновки

- Чітке розмежування шарів ізолює бізнес-логіку від протоколів передачі даних та деталей збереження.
- Spring IoC автоматизує життєвий цикл компонентів та зв'язування залежностей через інтерфейсні контракти.
- Впровадження колекцій реалізацій та шина доменних подій дозволяють нарощувати функціональність без модифікації існуючого коду.
- Використання конструкторів і незмінних залежностей спрощує ізольоване модульне тестування бізнес-шару без запуску повного контексту.

Індивідуальне завдання

Спроектувати та реалізувати шар бізнес-логіки для сутності власної предметної області:

- Дотримання тришарової архітектури: контролер взаємодіє із сервісом, а сервіс зі сховищем виключно через інтерфейсні контракти.
- Забезпечення незмінності стану сервісу (Stateless Singleton) та впровадження залежностей через конструктор.
- Реалізація щонайменше двох бізнес-правил (перевірка унікальності доменного ідентифікатора та валідація переходів скінченного автомата).
- Генерація типізованих доменних винятків при спробі порушення бізнес-правил.

Індивідуальне завдання

Вимоги до розширення бізнес-логіки та модульного тестування:

- Винесення варіативної частини бізнес-логіки в окремі компоненти патерну «Стратегія».
- Автоматичне впровадження колекції реалізацій стратегій через Spring DI без використання `@Qualifier`.
- Покриття позитивних та граничних сценаріїв швидкими модульними тестами на базі Mockito (`@ExtendWith(MockitoExtension.class)`).
- Ізольоване виконання тестів без завантаження повного контексту фреймворку.

Перевірка індивідуального завдання

1. Автоматизоване модульне тестування: запуск тестів за допомогою wrapper обраної системи збірки на вибір (`./mvnw test` для Maven або `./gradlew test` для Gradle) з результатом `BUILD SUCCESS` без завантаження повного контексту фреймворку.
2. Ручна перевірка API через виконання запитів утилітою `curl` :
 - Успішний POST-запит повертає HTTP 201 Created та дані з результатом роботи обраної бізнес-стратегії.
 - Спроба зареєструвати дублікат ресурсу повертає HTTP 409 Conflict, а неприпустимий перехід станів автомата відхиляється зі статусом HTTP 422 Unprocessable Entity у форматі `ProblemDetail` .

Групове завдання

Спроекувати та реалізувати архітектуру сервісного шару для ключових бізнес-сценаріїв командного проєкту:

- Абстрагування залежностей: взаємодія між компонентами виключно через інтерфейсні контракти.
- Забезпечення незмінності стану сервісів (Stateless Singleton) та впровадження залежностей виключно через конструктор.
- Побудова формальної моделі переходів станів доменної сутності з суворим контролем бізнес-правил.
- Генерація типізованих доменних винятків при спробі виконання некоректних операцій або переходів.

Групове завдання

Вимоги до патернів проєктування, модульності та тестування:

- Реалізація варіативних алгоритмів через патерн «Стратегія» з автоматичним впровадженням колекції Spring без `@Qualifier`.
- Послаблення зв'язності супутніх дій через асинхронну обробку доменних подій за допомогою `@ApplicationModuleListener` у Spring Modulith.
- Покриття всіх бізнес-сервісів швидкими модульними тестами на базі Mockito без запуску повного контексту фреймворку.
- Оформлення документації у файлі `README.md` з описом бізнес-правил та матриці переходів станів.

Перевірка групового завдання

1. Автоматизоване модульне тестування та архітектурна верифікація:

- Запуск модульних тестів за допомогою wrapper обраної системи збірки на вибір (`./mvnw test` для Maven або `./gradlew test` для Gradle) з результатом `BUILD SUCCESS` без завантаження повного контексту фреймворку.
- Виконання архітектурного тесту `ApplicationModules.of(...).verify()` для перевірки дотримання модульних кордонів та коректності реєстрації слухачів подій.

2. Ручна перевірка API та аудит архітектури:

- Взаємодія контролерів виключно з інтерфейсами сервісів, відсутність польового впровадження залежностей (`@Autowired`) та коректне повернення відповідей `ProblemDetail`.